

M15 TEACHING BOOK

Teacher-style, beginner-first, self-contained. External sources are visual/current support; this book owns the learning route.

Cloud rule: never confuse "I read the portal page" with "I proved the system." Save exact item names, run IDs, counts/totals, permissions tested, failure/recovery evidence and screenshots only from what you actually executed.

DE15-01 — Cloud fundamentals for data engineers

VIDEO FIRST

[Microsoft Learn DP-700 — Implement/manage \(Part 1\) — 03:07 onward — analytics-solution responsibilities and Fabric architecture context](#)

CURRENT OFFICIAL REFERENCE

[DP-700T00-A course](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Cloud data engineering replaces “one powerful laptop” with managed identity, storage, compute, orchestration and monitoring services. The engineering problem stays the same: know the input grain, preserve raw evidence, transform reproducibly, validate outputs and recover safely. Fabric is Microsoft's integrated analytics platform; OneLake is its organization-wide logical lake. Cloud convenience does not remove the need to reason about state, cost, permissions or failure.

Worked example — read it like English

Worked example - ParcelPulse receives parcel-status JSON from an API every 15 minutes. Land the unchanged response in Bronze, use a notebook to validate and create Silver, expose read-only tables through the SQL analytics endpoint, and monitor the pipeline. The service identity needs only the permissions required to read the source and write the intended target. Read it like English: receive -> preserve -> validate/transform -> serve -> observe. Each arrow changes state and therefore needs evidence.

Expected proof before you move on

Expected proof: one architecture map that labels source, persistent state, compute/orchestration, consumer, identity boundary and monitoring signal.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: If compute disappears tonight, which data/state must still exist tomorrow, and what evidence proves that?

DE15-01 — Guided → Solo → Validate

Guided reproduction

1. Open the normal learner_architecture_plan.json and leave every field blank first.
2. List API/files as sources and name one durable raw state in OneLake/lakehouse.
3. Add notebook/pipeline as compute/control flow and SQL/BI as consumer surfaces.
4. Add one identity boundary and one monitoring signal; then compare with EXPECTED proof, not a finished answer.

Guided expected check

Expected proof: one architecture map that labels source, persistent state, compute/orchestration, consumer, identity boundary and monitoring signal.

Independent task

Design a new Fabric architecture for a small parcel company with API + files + analytics consumers. Name state, compute, security and monitoring boundaries.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain cloud fundamentals for data engineers for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Design a changed Fabric architecture for a small parcel company with API + files + analytics consumers. Name state, compute, security and monitoring boundaries.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-02 — Identity, IAM and least privilege

VIDEO FIRST

[Microsoft Learn DP-700 — Implement/manage \(Part 1\) — Use the Implement/manage security/permissions portion; exact current security rules come from the linked OneLake docs](#)

CURRENT OFFICIAL REFERENCE

[OneLake security access-control model](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Identity answers who/what is calling; authentication proves that identity; authorization decides what it may do. Least privilege means grant only the minimum actions and data scope needed. In Fabric, workspace roles are control-plane permissions while OneLake security provides data-plane controls. A Contributor can create/modify many workspace items, so do not hand out workspace roles just to grant narrow data read access.

Worked example — read it like English

Worked example - An analyst only needs to query curated Gold data. Giving Contributor because it is convenient also grants broad workspace capabilities. A stronger design gives the analyst a narrow read path, gives the engineer the item/workspace rights needed to build, and gives the pipeline service identity only the source-read and target-write permissions it needs. Test with the intended identity; your own Admin success proves almost nothing about least privilege.

Expected proof before you move on

Expected proof: a permission matrix with at least one deliberately refused grant and one test performed as the intended narrow identity.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Why does successful access as Workspace Admin fail to prove least privilege?

DE15-02 — Guided → Solo → Validate

Guided reproduction

1. Create three rows: analyst, engineer, pipeline service identity.
2. For each, write only the minimum job it must perform.
3. Choose narrow workspace/item/data rights; write one tempting but excessive grant and refuse it.
4. Write how you would test the analyst and service identity separately.

Guided expected check

Expected proof: a permission matrix with at least one deliberately refused grant and one test performed as the intended narrow identity.

Independent task

Create a least-privilege permission matrix for a Viewer analyst, Contributor engineer and service identity; defend every grant.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain identity, iam and least privilege for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Create a least-privilege permission matrix for a Viewer analyst, Contributor engineer and service identity; defend every grant.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-03 — Azure Storage concepts and OneLake

VIDEO FIRST

[Microsoft Learn DP-700 — Implement/manage \(Part 1\) — 03:46 onward — workspace/storage context; then current OneLake reference](#)

CURRENT OFFICIAL REFERENCE

[OneLake documentation](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Azure storage concepts help explain Fabric: durable object/file storage is separated from compute. OneLake gives the tenant one logical lake namespace and can reference external data through shortcuts instead of copying it. The engineering choice is copy vs reference: copy when you need an independent governed snapshot/transform path; shortcut when live governed access without duplication is appropriate.

Worked example — read it like English

Worked example - A vendor owns a large governed Parquet dataset that changes hourly. If your team only needs governed live access, a shortcut can avoid copying bytes. If you must preserve an independent daily source snapshot for replay/audit, copy the landing data instead. Read it like English: shortcut = point to governed data where it lives; copy = create and own another physical version. Freshness, recovery and ownership decide the choice.

Expected proof before you move on

Expected proof: three copy-vs-shortcut decisions with freshness, ownership and replay/recovery reasons.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: What recovery requirement would make you choose copy even when a shortcut is technically possible?

DE15-03 — Guided → Solo → Validate

Guided reproduction

1. Take external Parquet, daily CSV and shared Delta source.
2. For each, write freshness, ownership and replay/audit needs.
3. Choose COPY or SHORTCUT from those requirements, not convenience.
4. Write one failure/recovery consequence for each choice.

Guided expected check

Expected proof: three copy-vs-shortcut decisions with freshness, ownership and replay/recovery reasons.

Independent task

Choose copy or shortcut for a changed source set and record one failure/recovery concern per choice.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain azure storage concepts and onelake for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Choose copy or shortcut for a changed source set and record one failure/recovery concern per choice.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-04 — Fabric workspace and item model

VIDEO FIRST

[Microsoft Learn DP-700 — Implement/manage \(Part 1\) — 03:46 onward — workspace configuration and item context](#)

CURRENT OFFICIAL REFERENCE

[Fabric workspace roles](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

A Fabric workspace is a collaboration/security boundary containing items such as lakehouses, notebooks, pipelines, warehouses and semantic models. Admin/Member/Contributor/Viewer are not “seniority levels”; they are permission bundles. Item type matters because each has a different execution/state model. Name workspaces and items so dev/test/prod intent is visible.

Worked example — read it like English

Worked example - Workspace de-retail-dev contains Lakehouse retail_lh, Notebook nb_silver_orders, Pipeline pl_orders, and a serving choice. The workspace is the collaboration/security boundary; each item owns a different kind of state. A notebook cell is not a pipeline dependency, and a workspace role is not a data-table contract. Your inventory should say what each item does, who owns it, and what proves it worked.

Expected proof before you move on

Expected proof: a workspace/item inventory naming purpose, owner, input/output and validation evidence for every item.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: What state belongs to the workspace, and what state belongs to an individual Fabric item?

DE15-04 — Guided → Solo → Validate

Guided reproduction

1. Create a dev workspace name that exposes environment and purpose.
2. Add lakehouse, notebook, pipeline and serving item one at a time.
3. For each item write input, output/state, owner and proof-of-success.
4. Check that no two items have the same vague responsibility.

Guided expected check

Expected proof: a workspace/item inventory naming purpose, owner, input/output and validation evidence for every item.

Independent task

Build a fresh workspace/item inventory from a blank template without looking at the guided example.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain fabric workspace and item model for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Build a fresh workspace/item inventory from a blank template without looking at the guided example.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-05 — OneLake paths, files, tables and shortcuts

VIDEO FIRST

[Microsoft Learn DP-700 — Ingest/transform \(Part 2\) — 01:49 onward — ingest/storage choices; then exact shortcut reference](#)

CURRENT OFFICIAL REFERENCE

[OneLake shortcuts](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

OneLake exposes lakehouse Files and Tables differently. Files can hold arbitrary lake data; Tables represent managed/discoverable Delta tables. Shortcuts point to data in another OneLake or supported external location without copying the bytes. A path that looks convenient can still be wrong if it bypasses the intended table contract or security boundary.

Worked example — read it like English

Worked example - Preserve source files under Files/bronze/orders/ingest_date=2026-09-10/ and publish conformed data as Tables/silver_orders. A shortcut at Files/reference/product_master can point to governed shared data without copying it. The SQL analytics endpoint naturally sees registered Delta tables, not every arbitrary file path. Use names that reveal layer and purpose rather than generic folders such as data1/final2.

Expected proof before you move on

Expected proof: explicit Files, Tables and shortcut paths plus a sentence explaining which bytes are copied and which are referenced.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Why can a path be technically reachable but still violate your intended table/security contract?

DE15-05 — Guided → Solo → Validate

Guided reproduction

1. Write a Bronze Files path that preserves ingest date.
2. Write a managed Silver table name and grain.
3. Add one shortcut for governed shared reference data.
4. State which path is visible as a Delta table to SQL and which is merely file/reference storage.

Guided expected check

Expected proof: explicit Files, Tables and shortcut paths plus a sentence explaining which bytes are copied and which are referenced.

Independent task

Design fresh OneLake Files/Tables/shortcut paths and explain how SQL/Spark would see each.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain onelake paths, files, tables and shortcuts for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Design fresh OneLake Files/Tables/shortcut paths and explain how SQL/Spark would see each.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-06 — Fabric Lakehouse and Delta tables

VIDEO FIRST

[Microsoft Learn DP-700 — Ingest/transform \(Part 2\) — 01:49 onward — batch transform/storage and lakehouse context](#)

CURRENT OFFICIAL REFERENCE

[Fabric lakehouse overview](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

A Fabric Lakehouse combines open Delta storage in OneLake with Spark-based engineering and a SQL analytics endpoint for read-oriented T-SQL access. Delta tables add transaction/version metadata around Parquet files. The lakehouse is not “a warehouse with folders”; it serves mixed structured/unstructured engineering while the Warehouse is optimized for T-SQL-centric relational workloads and supports different transaction capabilities.

Worked example — read it like English

Worked example - The SilverOrders table is one row per accepted order_id. Required columns are order_id, customer_id, event_date, state and value; bad date, missing customer and wrong business state are quarantined. Starting from the normal fixture, 7 raw rows become 4 Silver rows with value total 3300 before the incremental load. That count + total pair is a stronger checkpoint than a green notebook badge.

Expected proof before you move on

Expected proof: table grain + schema contract + bad-row rules + 7 raw / 4 accepted / 3 quarantined / 3300 control for the normal fixture.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Why are both accepted-row count and business total needed before trusting the Silver table?

DE15-06 — Guided → Solo → Validate

Guided reproduction

1. Open normal/base.csv and state raw row grain before cleaning.
2. Apply date/entity/business-state acceptance rules without changing raw source.
3. Reconcile 7 raw -> 4 accepted + 3 quarantined and accepted value 3300.
4. Write the SilverOrders table contract and one query/check you would use after write.

Guided expected check

Expected proof: table grain + schema contract + bad-row rules + 7 raw / 4 accepted / 3 quarantined / 3300 control for the normal fixture.

Independent task

Define a fresh lakehouse Delta table contract and validation for a new dataset.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain fabric lakehouse and delta tables for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Define a fresh lakehouse Delta table contract and validation for a changed dataset.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-07 — Fabric notebooks

VIDEO FIRST

[Microsoft Learn DP-700 — Ingest/transform \(Part 2\) — 01:49 onward — transformation choices; then notebook UI/reference](#)

CURRENT OFFICIAL REFERENCE

[How to use Fabric notebooks](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Fabric notebooks are browser-based code items used for Spark/Python/SQL workflows. Attach the intended lakehouse, make inputs/outputs explicit, separate parameters/config from business logic, and make reruns safe. A notebook that only works because cell 7 was manually run yesterday is not a production data pipeline.

Worked example — read it like English

Worked example - A reproducible notebook follows parameters/config -> read -> schema/business validation -> transform -> write -> reconciliation. If the notebook succeeds only because a variable was created manually in an earlier ad-hoc cell, it is stateful in the wrong way. Restart the session or rerun from the declared entry point to prove the notebook can reproduce its result.

Expected proof before you move on

Expected proof: rerunnable notebook flow with parameters/config separated from transformation logic and a clean-start validation plan.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: How would you prove a notebook does not depend on hidden manual cell history?

DE15-07 — Guided → Solo → Validate

Guided reproduction

1. Start with a blank notebook-flow list.
2. Add parameters/config before any data read.
3. Add read -> validate -> transform -> write -> reconcile in that order and mark which cells may have side effects.
4. Describe a clean-session rerun test and what evidence would fail it.

Guided expected check

Expected proof: rerunnable notebook flow with parameters/config separated from transformation logic and a clean-start validation plan.

Independent task

Design a rerunnable notebook flow from a blank file: parameters → read → validate → transform → write → reconcile.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain fabric notebooks for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Design a rerunnable notebook flow from a blank file: parameters → read → validate → transform → write → reconcile.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-08 — Spark in Fabric

VIDEO FIRST

[Microsoft Learn DP-700 — Ingest/transform \(Part 2\) — 01:49 onward — Spark transform context; current Fabric Runtime doc controls versions](#)

CURRENT OFFICIAL REFERENCE

[Apache Spark runtime in Fabric](#)

ADDITIONAL CURRENT REFERENCES

[Fabric Runtime 2.0](#)

Current GA runtime: Spark 4.1 / Python 3.13 / Java 21 / Delta 4.2

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Fabric Runtime is managed separately from upstream open-source Spark. As of this rebuild, Fabric Runtime 2.0 is GA with Spark 4.1, Java 21, Python 3.13 and Delta Lake 4.2. Our local M12/M13 course baselines are newer upstream versions. Therefore, test code against the runtime where it will execute; “Spark 4.x” alone is not a compatibility guarantee.

Worked example — read it like English

Worked example - Local course code was developed against Spark 4.2 and Delta 4.4, while Fabric Runtime 2.0 currently provides Spark 4.1 and Delta 4.2. Before moving code, check APIs used, package compatibility, SQL/function differences and Delta features. Read it like English: same major family does not mean same execution contract; the target runtime wins.

Expected proof before you move on

Expected proof: compatibility checklist that explicitly records local Spark 4.2/Delta 4.4 versus Fabric Runtime 2.0 Spark 4.1/Delta 4.2.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Why is “both are Spark 4.x” not enough evidence that local code is Fabric-compatible?

DE15-08 — Guided → Solo → Validate

Guided reproduction

1. Write LOCAL and FABRIC columns.
2. Record local Spark 4.2 / Delta 4.4 and Fabric Runtime 2.0 Spark 4.1 / Delta 4.2.
3. List APIs/packages/functions your project actually relies on.
4. For each dependency write a compatibility test or fallback before promotion.

Guided expected check

Expected proof: compatibility checklist that explicitly records local Spark 4.2/Delta 4.4 versus Fabric Runtime 2.0 Spark 4.1/Delta 4.2.

Independent task

Write a runtime-compatibility checklist for moving local Spark/Delta code into Fabric Runtime 2.0.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain spark in fabric for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Write a runtime-compatibility checklist for moving local Spark/Delta code into Fabric Runtime 2.0.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-09 — Fabric Data Pipelines

VIDEO FIRST

[Microsoft Learn DP-700 — Implement/manage \(Part 1\) — 17:00 onward — orchestration processes](#)

CURRENT OFFICIAL REFERENCE

[Fabric Data Factory — Copy job](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Fabric Data Pipelines orchestrate activities such as copy, notebook and other jobs. A pipeline is control flow: dependency, parameters, retries, failure routing and schedules. Data movement itself can use Copy activity/Copy job and other mechanisms. A pipeline is trustworthy only when a failed upstream activity prevents a misleading downstream publish and reruns are idempotent.

Worked example — read it like English

Worked example - Pipeline pl_orders has Copy Bronze -> Notebook Silver -> Validate -> Publish Gold. Publish depends on successful validation. A retry is safe only on activities whose side effects are idempotent or otherwise controlled. If Notebook Silver fails, the successful Bronze copy may remain, but Publish must not run and progress state must not pretend the batch is complete.

Expected proof before you move on

Expected proof: dependency graph, parameter, safe-retry decision, blocking failure route and checkpoint/publish rule.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Which successful upstream state may remain after a downstream failure, and what must stay unpublished?

DE15-09 — Guided → Solo → Validate

Guided reproduction

1. Draw Copy Bronze -> Notebook Silver -> Validate -> Publish.
2. Add a parameter such as load_date and identify durable checkpoint state.
3. Choose one retryable activity and explain why replay is safe.
4. Inject a Silver failure on paper: mark exactly which downstream activity is blocked.

Guided expected check

Expected proof: dependency graph, parameter, safe-retry decision, blocking failure route and checkpoint/publish rule.

Independent task

Design an independent pipeline with at least four activities, dependency, parameter, retry and failure branch.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain fabric data pipelines for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Design an independent pipeline with at least four activities, dependency, parameter, retry and failure branch.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-10 — SQL analytics endpoint and warehouse choice

VIDEO FIRST

[Microsoft Learn DP-700 — Ingest/transform \(Part 2\) — 01:49 onward — storage/serving choices; then exact SQL endpoint reference](#)

CURRENT OFFICIAL REFERENCE

[Lakehouse SQL analytics endpoint](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Every Fabric lakehouse automatically gets a SQL analytics endpoint: a read-only T-SQL surface over Delta tables. It is excellent for validation, ad-hoc analysis and downstream SQL consumption, but it is not the place to mutate lakehouse Delta data. Choose Warehouse when your workload is strongly relational/T-SQL-centric and needs warehouse capabilities; choose Lakehouse when Spark/open-file engineering is central.

Worked example — read it like English

Worked example - Use the lakehouse SQL analytics endpoint to SELECT counts/totals from Silver/Gold and serve read-oriented T-SQL consumers. Do not issue UPDATE against lakehouse Delta through that read-only endpoint. If a finance team requires T-SQL-first warehousing features and transactional warehouse behavior, evaluate Fabric Warehouse instead of forcing every workload through the lakehouse.

Expected proof before you move on

Expected proof: two read-only validation queries plus a justified lakehouse-vs-Warehouse choice for changed workloads.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: What makes SQL analytics endpoint a serving/validation surface rather than your general Delta mutation engine?

DE15-10 — Guided → Solo → Validate

Guided reproduction

1. Write a SELECT count and SUM validation query for Silver/Gold.
2. Mark them read-only and identify one UPDATE/MERGE operation that belongs on a write-capable path.
3. Classify three workloads as lakehouse endpoint or Warehouse.
4. Defend each classification using workload requirements rather than UI preference.

Guided expected check

Expected proof: two read-only validation queries plus a justified lakehouse-vs-Warehouse choice for changed workloads.

Independent task

Choose lakehouse SQL endpoint vs Warehouse for three changed workloads and defend each choice.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain sql analytics endpoint and warehouse choice for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Choose lakehouse SQL endpoint vs Warehouse for three changed workloads and defend each choice.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-11 — Medallion architecture in Fabric

VIDEO FIRST

[Microsoft Learn DP-700 — Ingest/transform \(Part 2\) — 01:49 onward — batch architecture context; then medallion tutorial](#)

CURRENT OFFICIAL REFERENCE

[Lakehouse end-to-end medallion scenario](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Medallion architecture is a contract pattern, not three folder names. Bronze preserves raw/landing truth, Silver applies validated/conformed business-ready structure, and Gold serves curated aggregates/products. Each layer needs a grain, acceptance rules, lineage and reconciliation. Do not silently “clean” Bronze until you can no longer reproduce what arrived.

Worked example — read it like English

Worked example - Bronze preserves all 7 normal raw rows, including bad records. Silver accepts 4 and quarantines 3. Gold aggregates only the accepted business-ready rows. Read it like English: Bronze answers what arrived; Silver answers what is valid/conformed; Gold answers what the consumer needs. Every layer states its grain and reconciles to the prior layer.

Expected proof before you move on

Expected proof: Bronze/Silver/Gold contracts with layer grain, acceptance/rejection behavior, lineage and reconciliation.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Which layer should preserve a malformed source record, and why should it not silently disappear?

DE15-11 — Guided → Solo → Validate

Guided reproduction

1. Keep all raw normal rows in Bronze including the three invalid business rows.
2. Apply acceptance rules to produce 4 Silver rows and a quarantine set.
3. Define one Gold aggregate at a stated grain.
4. Reconcile Bronze -> Silver+quarantine -> Gold and write the lineage sentence.

Guided expected check

Expected proof: Bronze/Silver/Gold contracts with layer grain, acceptance/rejection behavior, lineage and reconciliation.

Independent task

Define Bronze/Silver/Gold contracts for a changed domain with explicit grains and rejected-row handling.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain medallion architecture in fabric for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Define Bronze/Silver/Gold contracts for a changed domain with explicit grains and rejected-row handling.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-12 — Incremental loads and MERGE in cloud

VIDEO FIRST

[Microsoft Learn DP-700 — Ingest/transform \(Part 2\) — 00:47 onward — loading patterns: then current incremental-copy reference](#)

CURRENT OFFICIAL REFERENCE

[Fabric Copy job incremental copy](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Incremental loading moves only new/changed data after a known successful state. Watermarks, CDC and MERGE/upsert are mechanisms, not the goal. The safe rule is atomicity between published output and progress state: a failed load must not advance the checkpoint. Fabric Copy job can manage incremental state; notebook/Delta MERGE can implement explicit upsert logic when you need more control.

Worked example — read it like English

Worked example - Normal base state contains accepted O1, O2, O4 and O7. The next load updates O2 from 800 to 850 and inserts O8 for 900. Correct post-MERGE state is 5 rows / 4250, and an exact rerun changes nothing. If publish fails, the checkpoint stays at the last successful state so the same input can be replayed safely.

Expected proof before you move on

Expected proof: base controls, update/insert controls, 5 rows / 4250 after merge, exact-rerun no-op and failed-publish checkpoint rule.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Why must progress/checkpoint state not advance when publish fails?

DE15-12 — Guided → Solo → Validate

Guided reproduction

1. Run the bundled normal offline simulator and save `observed_controls.json`.
2. Explain O2 as update and O8 as insert; verify 5 rows / 4250 after merge.
3. Run it again and prove the same result does not create another side effect.
4. Describe an injected publish failure and write the prior checkpoint value that must remain durable.

Guided expected check

Expected proof: base controls, update/insert controls, 5 rows / 4250 after merge, exact-rerun no-op and failed-publish checkpoint rule.

Independent task

Design a new incremental load where a failed publish must not advance progress. State the atomic boundary.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain incremental loads and merge in cloud for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Design a changed incremental load where a failed publish must not advance progress. State the atomic boundary.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-13 — Permissions, connections and secrets

VIDEO FIRST

[Microsoft Learn DP-700 — Implement/manage \(Part 1\) — Use the Implement/manage security/connection portion; current OneLake security docs control permissions](#)

CURRENT OFFICIAL REFERENCE

[OneLake data security overview](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Connections centralize access to external systems and credentials. Never paste secrets into notebooks, Git, screenshots or logs. Separate workspace/item permissions from data access and from external-source credentials. OneLake security can scope data access more finely than a broad workspace role. Verify with a least-privilege test identity rather than assuming permissions from your own admin account.

Worked example — read it like English

Worked example - A notebook needs to read an external source. Store the connection/credential through the platform connection or secret mechanism, not as a literal token in the notebook. Give the pipeline identity only the necessary source and target access. Then test a narrow user who should read Gold but not modify engineering items. Security is proven by both allowed and denied behavior.

Expected proof before you move on

Expected proof: identity -> permission -> resource matrix, no literal secret, and evidence for one allowed plus one denied access path.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Name the three separate security questions: who are you, what may you do, and where/how is the external credential stored?

DE15-13 — Guided → Solo → Validate

Guided reproduction

1. Create identities for engineer, analyst and pipeline.
2. Separate workspace/item permissions, OneLake data permissions and external connection credential.
3. Replace any literal credential with a named connection/secret reference.
4. Write one allowed and one denied least-privilege test using a non-admin identity.

Guided expected check

Expected proof: identity -> permission -> resource matrix, no literal secret, and evidence for one allowed plus one denied access path.

Independent task

Find and repair three security mistakes in the changed plan: broad role, hard-coded secret and unclear external connection identity.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain permissions, connections and secrets for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Find and repair three security mistakes in the changed plan: broad role, hard-coded secret and unclear external connection identity.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-14 — Monitoring, reliability and cost awareness

VIDEO FIRST

[Microsoft Learn DP-700 — Monitor/optimize \(Part 3\) — 00:49 monitoring • 02:03 alerts • 10:01 optimization](#)

CURRENT OFFICIAL REFERENCE

[Fabric monitoring hub](#)

ADDITIONAL CURRENT REFERENCES

[Fabric pipeline pricing](#)

CU-based pipeline consumption and Capacity Metrics cost estimation

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Cloud reliability is observable behavior: run status, duration, freshness, row/value reconciliation, failure details and capacity consumption. Fabric Monitoring hub centralizes activity health; Capacity Metrics helps capacity administrators understand consumption. Cost awareness means selecting efficient load patterns and compute—not chasing the cheapest number while creating unreliable reruns.

Worked example — read it like English

Worked example - Run A finished but row count dropped unexpectedly; Run B failed authentication three times; Run C succeeded but duration/capacity consumption doubled. Monitoring asks different first questions: data-quality/reconciliation for A, identity/credential for B, and workload/capacity/plan for C. A dashboard is useful only when it leads to a diagnostic and recovery action.

Expected proof before you move on

Expected proof: monitoring record covering run status, freshness, quality/reconciliation, duration, failure detail and capacity/cost signal.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: Why can “Succeeded” still be an operationally bad run?

DE15-14 — Guided → Solo → Validate

Guided reproduction

1. Create three run records: quality anomaly, auth failure, duration/capacity spike.
2. For each record write the first evidence you inspect.
3. Choose owner + recovery action + downstream validation.
4. Add freshness and cost/capacity signal so success/failure is not the only telemetry.

Guided expected check

Expected proof: monitoring record covering run status, freshness, quality/reconciliation, duration, failure detail and capacity/cost signal.

Independent task

Create a monitoring checklist covering freshness, quality, failure, duration and capacity/cost signals.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain monitoring, reliability and cost awareness for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Create a monitoring checklist covering freshness, quality, failure, duration and capacity/cost signals.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-15 — Git, deployment and environment separation

VIDEO FIRST

[Microsoft Learn DP-700 — Implement/manage \(Part 1\) — 06:22 onward — lifecycle management in Fabric](#)

CURRENT OFFICIAL REFERENCE

[Fabric CI/CD overview](#)

ADDITIONAL CURRENT REFERENCES

[Get started with Fabric Git integration](#)

Connect workspace to Git and sync lifecycle

[Fabric lifecycle best practices](#)

Dev/Test/Prod lifecycle guidance

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

Git integration versions supported Fabric item definitions; deployment pipelines promote content through environments. Dev/Test/Prod should differ through configuration, not manual edits inside copied notebooks. Source control is not deployment by itself. A safe release has review, environment-specific variables/connections, validation after deploy, and a rollback/recovery plan.

Worked example — read it like English

Worked example - Git stores the supported Fabric item definitions. Dev deploys from the working branch, Test validates the promoted build with test connections, and Prod receives an approved deployment using production configuration. Never edit a production notebook just to fix a dev path. Configuration differs by environment; business logic should be promoted deliberately, with rollback evidence.

Expected proof before you move on

Expected proof: dev/test/prod mapping, Git branch/review path, environment configuration separation, validation-after-deploy and rollback plan.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: What belongs in environment configuration instead of being manually edited inside promoted code?

DE15-15 — Guided → Solo → Validate

Guided reproduction

1. Name dev, test and prod workspaces.
2. Map Git branch/review and deployment stage to those environments.
3. Move paths/connections/IDs into environment-specific configuration.
4. Write validation-after-deploy and rollback evidence before calling promotion safe.

Guided expected check

Expected proof: dev/test/prod mapping, Git branch/review path, environment configuration separation, validation-after-deploy and rollback plan.

Independent task

Design a Git + deployment pipeline flow with dev/test/prod configuration separation and rollback evidence.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain git, deployment and environment separation for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Design a Git + deployment pipeline flow with dev/test/prod configuration separation and rollback evidence.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.

DE15-16 — End-to-end Fabric data engineering case

VIDEO FIRST

[Microsoft Learn official DP-700 video playlist — Use the full official DP-700 playlist as stage review; execute the course case rather than only watching](#)

CURRENT OFFICIAL REFERENCE

[DP-700T00-A end-to-end course](#)

Close the external source before the independent task. If the video changes or disappears, the Teaching Book + course files remain sufficient for the lesson.

Teacher explanation

An end-to-end Fabric engineer must connect architecture, access, ingestion, lakehouse storage, Spark/notebook transformation, orchestration, SQL serving, monitoring and lifecycle management. The final proof is not “I clicked every menu.” It is a reproducible system with evidence: what arrived, what was accepted/rejected, what changed incrementally, what ran, who could access it, and how you recovered from failure.

Worked example — read it like English

Worked example - Retail source -> Bronze raw -> Silver validated -> Gold serving, orchestrated by a pipeline and transformed by a notebook, validated through the SQL endpoint, governed by least privilege, monitored for freshness/failure/cost, and promoted through dev/test/prod. A failure drill must show which state stays durable, what does not publish, how rerun works, and which exact evidence proves recovery.

Expected proof before you move on

Expected proof: end-to-end architecture/evidence index that distinguishes actual Fabric observations from offline design/simulator evidence.

Common beginner traps

- Copying portal steps without understanding the object/state created.
- Assuming an Admin experience equals a least-privilege user experience.
- Treating a successful run badge as enough without data reconciliation.
- Hard-coding workspace IDs, paths, credentials or environment-specific values.

STOP — check your understanding

STOP: If the final Gold output is wrong, what evidence lets you trace backward to the exact failed boundary?

DE15-16 — Guided → Solo → Validate

Guided reproduction

1. Start from the normal blank architecture plan, not the worked example.
2. Connect source -> Bronze -> Silver -> Gold -> serving, with notebook and pipeline responsibilities separated.
3. Add identity/security, monitoring and lifecycle controls.
4. Index the evidence for data controls, failure/recovery and every step actually executed vs only designed offline.

Guided expected check

Expected proof: end-to-end architecture/evidence index that distinguishes actual Fabric observations from offline design/simulator evidence.

Independent task

Produce the complete normal-domain architecture/evidence plan from a blank template and explain it without the book open.

Validate

Save one structural proof (item/path/permission/config), one data/run proof (count, total, status or version), and one explanation of how the result could be wrong. For UI work, capture only what you actually observed. For offline work, label it OFFLINE PLAN/SIMULATOR.

Recover

Preserve the failed/weak attempt. Inspect the first incorrect boundary: identity/permission, path/item, input grain, transform, state/checkpoint, or serving/monitoring. Open the protected Review only after the saved attempt exists.

Explain back

Explain end-to-end fabric data engineering case for 60–90 seconds. Name the object, the state it controls, one failure mode and one validation.

Fresh changed retry

Produce the complete clinic retest architecture/evidence plan from a blank template and explain it without the book open.

Mastered only when

You can complete the changed task without Review open and defend your actual evidence. A percentage score never substitutes for missing runtime or permission proof.